

alembic demo

the model

```
schema:
  types:
    dcim.site:
      key:
        slug: { type: slug }
      fields:
        name: { type: string }
        slug: { type: slug }
        status: { type: string }
    dcim.device:
      key:
        name: { type: slug }
      fields:
        name: { type: string }
        site: { type: ref, target: dcim.site }
        role: { type: ref, target: dcim.device_role }
        device_type: { type: ref, target: dcim.device_type }
        status: { type: string }
```

objects reference each other by UID

```
objects:
- uid: "100000000-...-0000000000001"
  type: dcim.site
  key: { slug: "fra1" }
  attrs:
    name: "FRA1"
    slug: "fra1"
    status: "active"

- uid: "100000000-...-0000000000005"
  type: dcim.device
  key: { name: "leaf01" }
  attrs:
    name: "leaf01"
    site: "100000000-...-0000000000001"
    status: "active"
```

no backend IDs. references are stable UUIDs.

validate

```
$ alembic validate -f core-v1.yaml  
ok
```

offline, no backend needed.

checks schema, keys, UID uniqueness, and ref integrity.

plan

```
$ alembic plan -f core-v1.yaml -o plan.json  
plan: 5 to create, 0 to update, 0 to delete  
plan written to plan.json  
[{ "op": "create", "count": 5 }]
```

deterministic diff: desired model vs observed backend state.

apply

```
$ alembic apply -p plan.json --interactive
create dcim.site {"slug":"fra1"}? [y/N] y
create dcim.device_role {"slug":"leaf"}? [y/N] y
create dcim.manufacturer {"slug":"acme"}? [y/N] y
create dcim.device_type {"slug":"leaf-48p"}? [y/N] y
create dcim.device {"name":"leaf01"}? [y/N] y
applied 5 operations
```

each op is a separate API call. state store tracks UID-to-backend-ID mappings.

evolve the model

two changes from v1:

- ▶ **add** leaf02
 - ▶ **update** leaf01 status: active -> staged
- ```
- uid: "10000000-...-0000000000005"
 type: dcim.device
 key: { name: "leaf01" }
 attrs:
 status: "staged" # was "active"
```
- ```
- uid: "10000000-...-0000000000006"  
  type: dcim.device  
  key: { name: "leaf02" }    # new  
  attrs: ...
```

re-plan: only the delta

```
$ alembic plan -f core-v2.yaml -o plan.json  
plan: 1 to create, 1 to update, 0 to delete  
plan written to plan.json
```

same tool, same workflow. alembic only touches what changed.

custom types

```
virt.hypervisor:
  key:
    name: { type: slug }
  fields:
    name: { type: string }
    site: { type: ref, target: dcim.site }
    host_device: { type: ref, target: dcim.device }
    platform: { type: string }
virt.vm:
  key:
    name: { type: slug }
  fields:
    hypervisor: { type: ref, target: virt.hypervisor }
    cpu: { type: int }
    memory_mb: { type: int }
```

no plugin needed. your schema, your types.

validate custom types

```
$ alembic validate -f extended-model.yaml  
ok
```

target a backend that supports your types, or scaffold one with `alembic cast django`.